

SOFTWARE REPORT

CAVY

Codebase, Frameworks, Code Quality & NFR Assessment

A desktop and classroom platform for programming education. Students complete coding labs with an AI assistant available while the platform records a structured trail of how the assistant was used; professors and an administrator manage classes on a shared server.

SOURCE (PYTHON)	FRONT END (JS)	TESTS	COVERAGE	LINT / TYPE / SECURITY
9,825 lines 44 files	5,467 lines	457 pass 7,158 lines	89.0%	0 / 0 / 0

Tools: Ruff, mypy (strict), Radon, Bandit, pip-audit, pytest + coverage, mutmut. Generated 08 Oct 2026 by `scripts/make_audit_report.py`, which runs every tool against the current source tree; nothing here is copied from an earlier report.

1. Codebase

1.1 What CAVY is

CAVY runs programming labs where an AI assistant is available throughout, and treats the assistant as part of the system, not a black box: every prompt, reply, code edit, run and submission is stored as an event in the same database the rest of the platform reads. Everything else (the CIQ score, the progress charts, the professor's reports) is derived from that event log.

It runs in two ways from the same code: standalone on one computer (its own SQLite file), or against a central classroom server that a professor and many students on one network share live, with a browser admin panel. Installers are built for macOS and Windows.

1.2 Architecture and modules

Module	Lines	Responsibility
db/	2615	SQLAlchemy models, the engine (WAL, pooling), accounts, courses, resources, approvals
server/	2111	FastAPI classroom server, onboarding by email, the browser admin panel
api/	1982	CavyApi : the one bridge the web front end talks to (every operation)
client/	872	The app's side: local data or a server, lab-mode lock-down, remote calls
ai/	777	AIProvider interface; Ollama and hosted providers (OpenAI, Claude, Gemini, Grok, Groq)
signals/	670	The 14 CIQ signals computed from the event log
events/	141	Background, batched, append-only event logger
sandbox/	138	Runs student code in a separate process with a time limit

1.3 How the parts connect

The window (HTML/CSS/JS in a native web view, Monaco editor) calls a single Python object, **CavyApi**, for everything. On a server, the same CavyApi class is reused unchanged: the server creates one per signed-in person and exposes its methods over HTTP, so the app, the admin panel and the tests all exercise the same code. A change feed (long polling) tells open screens when something they show has changed, so a professor's upload appears on a student's screen without a refresh.

Students' code never runs inside the app: each run is a separate process with a time limit and, where the operating system supports it, CPU and memory limits. The event logger writes to the database on its own thread so typing never waits for the disk.

1.4 Reproducible setup

`requirements.lock` pins 151 packages to the exact versions the tests run against; `pyproject.toml` declares the ranges and optional groups (server, build, dev). The README covers setup, running, the quality gate, building installers and the architecture, and `docs/` holds the classroom guide, performance results and roadmap.

2. Frameworks, with the alternatives considered

Need	Chosen	Why this one	Alternatives weighed
Desktop shell	pywebview	Uses the system's web view (WebKit / WebView2), so the installer is ~70-80 MB, not an Electron-sized bundle; a plain Python object can be called straight from JavaScript.	Electron (bigger, adds Node), Tauri (needs Rust), PySide6 (the first version; slower to build UI and style).
Server	FastAPI + uvicorn	Typed request models, easy testing with TestClient, a small footprint, and it can serve the admin panel's static files.	Django (heavier, would duplicate the CavyApi layer), Flask (no built-in validation).
Database	SQLite (WAL) + SQLAlchemy 2	Zero setup for a classroom server or a laptop; WAL lets the event thread write while requests read; SQLAlchemy keeps the schema in typed models and could move to Postgres.	PostgreSQL (extra install, overkill for one class), plain sqlite3 (no models/migrations help).
Code editor	Monaco	The editor VS Code uses: Python highlighting, indentation, find; a plain-textarea fallback keeps the app working without it.	CodeMirror (lighter, fewer features), a bare textarea (no indenting).
AI access	httpx + a provider interface	One small HTTP client; each company is a short class behind the same interface, so a new provider is ~30 lines and tests use a mock transport.	Each company's own SDK (4 extra dependencies that change often), LangChain (far heavier).
Installers	PyInstaller, Inno Setup, dmgbuild, GitHub Actions	One recipe builds each OS's app; Actions builds and smoke-tests both installers on clean machines.	py2app / briefcase (one OS or less mature), WiX (heavier than Inno Setup).
Quality tools	Ruff, mypy strict, pytest, Radon, Bandit, pip-audit, mutmut	Each covers one risk: style and bugs, wrong types, behaviour, complexity, security patterns, vulnerable packages, and whether tests really check results.	Flake8 + isort + pyupgrade (Ruff replaces all three in one fast tool).

Dependencies are kept small on purpose (no front-end build step, no ORM add-ons, no task queue); the lock file lists what is actually installed and pip-audit checks it.

3. Code quality

Check	Result
Ruff (lint + format)	0 issues across src and tests
mypy --strict	0 errors in ? source files
Radon complexity	622 blocks, average grade A (3.23). A: 507, B: 101, C: 14, D or worse: 0. Most complex: <code>_apply</code> in <code>db/resources.py</code> (13).
Radon maintainability	41 of 44 files grade A, 2 B, 1 C. Lowest: <code>api/bridge.py</code> (0.0, C), <code>db/academics.py</code> (16.0, B), <code>server/admin.py</code> (18.5, B).
pytest	457 passed, 1 skipped; line+branch coverage 89.0% (floor enforced at 70%)
Bandit	0 high, 0 medium, 0 low over 9,569 lines; 18 deliberate suppressions, each with its reason in the code
pip-audit	No known vulnerabilities in the 151 pinned packages
Mutation testing	mutmut on <code>signals/compute.py</code> (the scoring engine): killed=876 survived=175 timeout=0

What the numbers do not hide. The 14 blocks graded C are in the places that branch on many cases (spreadsheet import, resource validation, account set-up); none is above 13. `api/bridge.py` (about 2,000 lines, one class) has a poor maintainability index purely from size, and is the first thing to split into smaller modules. Window-creation code (`app.py`) and the operating-system lock-down can only be tested on real screens, so they have little automated coverage.

4. Non-functional requirements

Area	Requirement	How it is met / measured
Performance	A class works at once without stalls	Measured (scripts/benchmark.py, docs/PERFORMANCE.md): 30 students at once, about 800 calls/s, 0 failures, typing/saving ~4 ms median; 100 students at once, 0 failures. The benchmark found a connection-pool limit that stalled requests 30 s at ~15 students; fixed and covered by tests.
Performance	The UI never waits on the database	Event logger on its own thread, batched writes (~38,000 events/s); SQLite WAL.
Reliability	Student code cannot hang or crash the app	Every run is a separate process with a wall-clock time limit; output is capped.
Reliability	No work lost; predictable failures	Autosave on every edit and on leaving a stage; events flushed before scoring and on shutdown; unexpected server errors return a plain message and are logged; a signal with no evidence reports <i>none</i> , never an invented value.
Logging	Problems can be investigated later	Structured JSON-lines logging to a rotating file (app + server), with an audit log of every sign-in, account change and admin action, also stored in the database.
Security	Passwords and secrets	Passwords: salted PBKDF2-SHA256 (200,000 rounds). Students' AI keys stay in the app's memory and are never sent to the server. The mail password is stored on the server only and never shown back. Admin passwords are replaced, never read.
Security	Access control and input	Roles (student, professor, administrator) enforced on the server; only approved emails can register; login throttling (admin) and request limits (emailed codes); uploads limited by size and type, links limited to http(s); emailed reset codes expire in 15 minutes and allow 5 tries; Bandit finds nothing.
Security	Known gaps	Traffic is plain HTTP on the local network (no HTTPS yet); installers are not code-signed; lab mode cannot block every operating-system escape (e.g. Ctrl+Alt+Del).
Maintainability	Typed, linted, tested	mypy strict and Ruff clean; coverage above; pre-commit runs them on every commit; CI runs the tests on Linux and macOS (the Windows test job on GitHub currently does not finish and is still being investigated; the Windows installer itself is built and smoke-tested).
Portability	macOS and Windows	One code base; two installers built and smoke-tested by GitHub Actions; per-OS data folders; per-OS lab-mode lock-down.
Usability	People can set it up	Guides in docs/ (classroom set-up, Windows testing), one-click admin-password script, emailed first logins, plain-language error messages.

Overall assessment

#	Section	Assessment	Honest rating
1	Codebase	Clear packages around one event-sourced database; a single bridge class as the front-end boundary; reproducible install. Weak spot: <code>bridge.py</code> is too big.	Good
2	Frameworks	Each choice is tied to a constraint and compared with alternatives; dependencies are few and pinned. The comparison is reasoning, not benchmarks.	Good
3	Code quality	Ruff, mypy strict and Bandit clean; 89% coverage; no function above 13 complexity; mutation score recorded above.	Good
4	NFRs	Performance now measured; logging added; security basics done; HTTPS, signing and validated scoring weights are still open.	Good, with listed gaps

Next steps, in order: split `api/bridge.py` into modules; HTTPS for the classroom server; code-signing the installers; run mutation testing on more than the scoring engine; test lab mode on more computers.

Report prepared by Group 04.